

Questbase Security Audit

Full review of the API surface, Supabase authorization model, browser rendering paths and delivery controls — verified against live production state, not only the repository. The headline: the security posture is genuinely strong, one control is quietly not doing its job, and the data model will not survive the first real customer.

SCOPE	LIVE SOURCES	FINDINGS	SYSTEM STAGE
26 API handlers • 3 Edge Functions • 147 migrations • 199 source modules	Supabase advisors, catalog & grants; npm audit; repo at e25dcc1	6 vulnerabilities • 6 hardening • 7 operational	Pre-launch pilot — 14 companies, 13 users, near-zero business data

Verdict

I could not find a way to read another tenant's data. Every tenant table has RLS enabled with at least one policy, no SECURITY DEFINER function is reachable by anonymous callers, and every untrusted-input render path escapes correctly. What follows is mostly about controls that are one deployment away from mattering, and about scale.

O DEPENDENCY CVES PROD & DEV	O ERROR-LEVEL SUPABASE ADVISORS	O ANON-EXECUTABLE DEFINER FUNCTIONS	O FUNCTIONS WITH MUTABLE SEARCH_PATH
7 TENANT TABLES MISSING FROM THE CI GUARD	43 UNBOUNDED QUERIES PER PAGE LOAD	525kB LARGEST BUILDER DOC ON A TEST COMPANY	~75 TABLES WITH LEGACY ANON DML GRANTS

Vulnerabilities

SIX • NONE PREVIOUSLY RECORDED

Ordered by how close each is to being exploitable today. None require a compromised account; all are reachable from a public or low-privilege position.

S1

MEDIUM

Client Portal

Revoking a portal link does not end sessions already open on it

`client-portal-open.js` checks `status=eq.active` and mints a six-hour HMAC session. The five session-authenticated endpoints — document URL, document file, document status, annotations, export event — verify only the signature and expiry. **None re-reads `client_portals.status`.**

So the response to a leaked portal link is not actually a response. A guest who opened the portal before you revoked it keeps downloading job documents for up to six hours, and nothing in the product tells you that.

FIX

Re-check portal status inside the `auth: 'portal-session'` branch of `_lib/endpoint.js`. One place, all five endpoints. Add a `sessions_valid_after` column so revocation is instant rather than waiting out the token.

S2

MEDIUM

Client Portal

A guest's display name is what proves they own their own markups

In `client-portal-annotations.js`, ownership is expressed as a filter on the guest's chosen name, and that name is free text supplied at portal-open time:

```
const guestName = String(session.guest_name || 'Guest').slice(0, 80);
... &guest_name=eq.${encodeURIComponent(guestName)}', { method: 'DELETE' }
```

Anyone holding the portal link can reopen it as “Sarah” and inherit Sarah's annotations. Posting an empty `annotations` array then deletes every markup under that name for the document. The single-annotation path defends partially — it preserves the original author and merges only the comment thread — but that itself lets any guest append arbitrary replies to someone else's markup.

FIX

Mint a random `guest_id` into the signed session at open time and scope ownership on that. Keep `guest_name` for display only.

S3

MEDIUM

Intake links

The intake passcode lockout can be walked past with parallel requests

`wb-intake-open.js` reads `failed_attempts` off the row it already loaded, adds one, and writes it back. Eight simultaneous wrong guesses all read the same value and all write the same `1`, so `MAX_PASSCODE_ATTEMPTS` never trips.

This matters more than the arithmetic suggests, because the file's own comment names this control as the one carrying the weight: *“the per-link lockout below is the real defence — this limiter is in-memory and per serverless instance.”* The backstop is the thing that isn't holding. A six-character passcode over a 31-character alphabet is ~887M combinations, so this is expensive rather than instant — but it is not the control that was designed.

The same read-modify-write appears on `submission_count` in `wb-intake-submit.js`, so a link capped at one submission accepts several posted together.

FIX

Move both counters into an atomic statement — `set failed_attempts = failed_attempts + 1 returning ...` — behind a `SECURITY DEFINER` function.

S4

MEDIUM

Cost / abuse

`/api/address-suggestions` is unauthenticated and bills to your Google account

No bearer token, no origin check, no membership check — the only endpoint in the whole API that reads no session at all. It proxies Google Places Autocomplete using `GOOGLE_MAPS_API_KEY`, falling back to Nominatim.

The only brake is the in-memory limiter, so the real ceiling is 60/min *multiplied by live lambda instances* and reset by every cold start. Anyone who reads the network tab once can drive your Places billing, or get your Nominatim access blocked for exceeding their usage policy.

FIX

It is only ever called from signed-in forms — require the bearer token, add `requireOrigin: true`, and put the counter somewhere durable.

S5

DORMANT

SMS

SMS send authorizes at company scope, crossing the workspace boundary

`sms-send.js` checks `isActiveMember(db, contact.company_id, user.id)` and nothing else. Your stated invariant is that `workspace_id` is the operational boundary — so a member of Workspace A can text a contact belonging to Workspace B.

Workspace SMS is disabled is already in your known-issues. What isn't recorded is that the endpoint would authorize across workspaces the moment it wakes up, and that the gate keeping it asleep lives in a *different file* — a hardcoded

`WORKSPACE_SMS_BACKEND_VERSION = 0` in `sms-readiness.js`. Setting a provider key is the whole distance between here and a live cross-workspace path.

Related: `sms-inbound.js` creates contacts with **no `workspace_id` at all**, and refetches every contact in the company on each inbound message to match a phone number in JavaScript.

FIX

Either complete the workspace-safe migration or delete the two endpoints until you do. A deployed endpoint held shut by a constant in another module is a fragile way to keep a feature off.

S6

LOW

Secrets

Portal session signing falls back to the service-role key

```
const sessionSecret = () =>
  process.env.CLIENT_PORTAL_SESSION_SECRET || supabaseServiceKey();
```

If `CLIENT_PORTAL_SESSION_SECRET` isn't set in production, the most privileged credential in the system doubles as the portal token signing key. HMAC doesn't leak its key, so this isn't directly exploitable — but it couples service-key rotation to invalidating every portal session, and it puts the crown jewel into a code path it has no business being in.

FIX

Confirm the variable is set in Vercel production, then make the fallback throw there instead of silently substituting. Keep the dev convenience behind a `NODE_ENV` check.

Hardening VERIFIED LIVE · NO CURRENT EXPOSURE

None of these is exploitable today. Each removes a layer you are currently relying on a single other layer to cover.

H1

DEFENSE IN DEPTH

Postgres grants

The `anon` role still holds blanket DML grants on ~75 legacy tables

Live check: `anon` has `SELECT, INSERT, UPDATE, DELETE, TRUNCATE` on essentially every table created before August — `companies`, `contacts`, `jobs`, `profiles`, `company_memberships`, `roles`, `role_permissions`. Your newer tables (`wb_intake_*`, `sms_*`, `record_history`, `ringcentral_*`) correctly have none, so the practice changed — the old grants were just never revoked.

RLS is currently the only barrier, and I confirmed it holds: every tenant table has RLS on with at least one policy, every policy fails closed on a null `auth.uid()`, and PostgREST never issues `TRUNCATE`. The cost is that a single future table shipped without RLS is instantly world-writable rather than merely misconfigured.

FIX

`REVOKE ALL ON ALL TABLES IN SCHEMA public FROM anon;` then grant back only what the public form, proposal and portal flows genuinely read.

H2

CONSISTENCY

Auth kernel

Two of the four authorization functions are hardened differently from the other two

<code>is_company_admin</code>	<code>SET search_path = ''</code>	✓ <code>auth.uid()</code>
<code>IS NOT NULL</code>		
<code>has_workspace_permission</code>	<code>SET search_path = ''</code>	✓ <code>auth.uid()</code>
<code>IS NOT NULL</code>		
<code>is_company_member</code>	<code>SET search_path = 'public','pg_temp'</code>	implicit
<code>only</code>		
<code>has_company_permission</code>	<code>SET search_path = 'public','pg_temp'</code>	implicit
<code>only</code>		

These four are the authorization kernel for the entire tenant model. `pg_temp` in a `SECURITY DEFINER` search path is the classic shadowing hazard; it's mitigated here because references are schema-qualified and `pg_temp` is last, and the two without an explicit null check still fail closed through `NULL = NULL`. But there is no reason for the same four functions to be hardened two different ways.

FIX

Bring the lower two up to `search_path = ''` with an explicit `auth.uid()` `is not null`, matching the pair that already has it.

H3

CONSISTENCY

RLS policies

Newer tables' policies target `T0 public` instead of `T0 authenticated`

`company_contacts`, `company_contact_fields`, `company_contact_options`, `wb_intake_links`, `wb_intake_submissions`, `wb_record_events`, plus the `company_memberships` read and `profiles` update policies. The predicates fail closed for anonymous callers, so there is no live exposure — but the rest of the schema carries a role-level backstop these don't.

`wb_intake_links` is the one to care about: it holds intake passcode hashes and salts under a `T0 public ... ALL` policy.

FIX

Recreate these policies `T0 authenticated`. Behaviour is unchanged; the backstop returns.

H4

PRIVACY

CSP

Customer documents are rendered through Microsoft's servers

The enforced CSP includes `frame-src ... https://view.officeapps.live.com`. Opening an Office document in the portal transmits it to Microsoft. That's a normal engineering trade — but it is a subprocessor relationship, and your customers are roofing companies handling homeowner contracts and insurance paperwork.

Separately, and already in your known-issues: the enforced policy retains `'unsafe-eval'`, and two jsPDF export paths are currently broken by the policy that shipped in `ca0839e`.

FIX

No code change — a subprocessor entry and a line in the privacy policy. Worth settling before the first enterprise security questionnaire arrives.

H5

POLICY

Roles

`construction_supervisor` counts as an admin for billing and call data

It sits in `ADMIN_ROLES` in `_lib/user-auth.js`, which gates the RingCentral proxies, and in `BILLING_ROLES` in `create-checkout-session.js`. So a construction supervisor can open a Stripe checkout for the company and read company-wide call records.

This may well be intended. It is worth being intended *on purpose* — the role name does not suggest either capability, and both lists are easy to miss when roles change.

FIX

Confirm, then move both to a named permission key rather than a hardcoded role list.

H6

CONFIG
Origins

The default app origin is still the gamma deployment

`DEFAULT_APP_ORIGIN` in `_lib/http-security.js` is `quest-hq-command-center-gamma.vercel.app`, and `safeReturnUrl()` falls back to it. A Stripe checkout whose `return_url` fails validation returns a paying production customer to the gamma build.

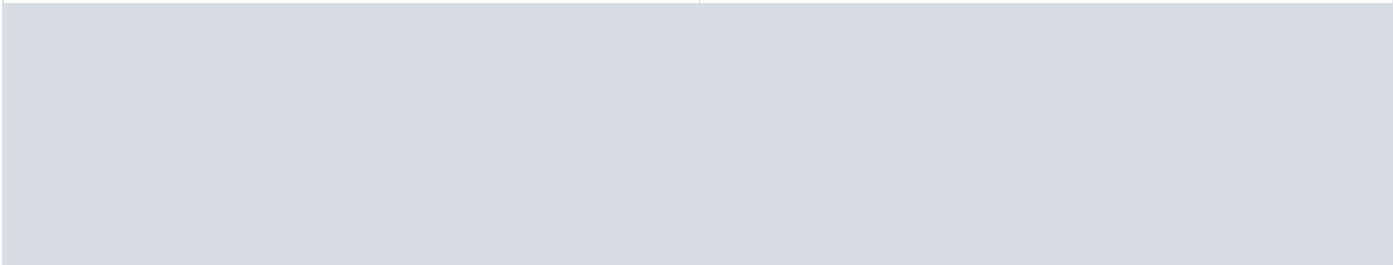
The invite Edge Function already solved this correctly — it defines a separate `CANONICAL_APP_URL` of `https://www.questbase.io` precisely because a deployment host is the wrong permanent address. That reasoning applies here too.

FIX
Point `DEFAULT_APP_ORIGIN` at `https://www.questbase.io` and keep gamma in the configured `APP_ORIGINS` list.

What's working

Stating these explicitly because they're load-bearing, and because an audit that only lists problems misrepresents the system.

✓ Tenant isolation holds Every tenant table has RLS enabled with at least one policy — verified against the live catalog, not the snapshot. No cross-tenant read path found.	✓ No anon-reachable privileged code Zero <code>SECURITY DEFINER</code> functions in <code>public</code> are executable by <code>anon</code>, and zero have a mutable <code>search_path</code>.
✓ The escaping discipline is real Traced every untrusted-input render path — SMS bodies, form answers, intake values, portal guest names, chat mentions. All route through <code>h()</code>. Even <code>linkifyMentions</code> escapes before it linkifies.	✓ Stripe webhook verification is textbook Timing-safe comparison, timestamp tolerance, hex-shape validation, and no internal error text echoed to the caller.
✓ <code>defineEndpoint</code> is the right pattern A single writer for every response means security headers cannot be skipped by a new handler. Four legacy endpoints still bypass it; the other twenty-two can't get it wrong.	✓ Clean supply chain and secrets Zero npm vulnerabilities across prod and dev. No committed secrets, no baked-in default credentials, <code>.env.local</code> correctly ignored.



✓ Public input validation is careful

Upload allowlists check every filename segment, block SVG and HTML by name, and reject ZIP-based Office formats deliberately. Intake field projection withholds internals rather than filtering them client-side.

✓ A stale known-issue is actually fixed

The note about `touch_eod_report_updated_at()` being anon-executable no longer reflects live state. Worth closing.

Operational findings WHERE THE SYSTEM WILL BEND FIRST

01

CRITICAL CONTROL

CI gate

The tenancy guard has been passing green on tables it has never checked

`npm run tenancy:check` runs inside `npm run check`, your pre-deploy gate. It reads `.ai/database/snapshot.json` — generated **2026-08-10**. Seven tenant-carrying tables now in production are absent from that file entirely:

<code>company_contacts</code>	<code>wb_intake_links</code>
<code>company_contact_fields</code>	<code>wb_intake_submissions</code>
<code>sms_messages</code>	<code>wb_record_events</code>
<code>sms_numbers</code>	

The script iterates the snapshot, so a table it doesn't know about is silently skipped rather than flagged. It has been reporting success on a set that no longer matches production. I checked all seven against the live catalog and they are correct — so nothing is broken. **The control is what's broken**, and it is the control the whole multi-tenant safety argument rests on.

FIX

Make `ai:check` fail when the newest migration filename postdates `snapshot.generated_at`. Better: drive the check from the live catalog so it cannot drift at all.

02

SCALE

Startup path

Every page load downloads the entire tenant

`loadInitialDataQueries` fires **43 concurrent queries**, most of them `select('*')` with no limit: jobs, tasks, contacts, deals, accounts, sites, job files, company contacts, calendar events, invites, profiles, and the whole workspace builder document.

This is invisible today — 14 companies, 13 users, single-digit business rows. It does not survive the first real customer. A roofer with 20,000 contacts and five years of jobs downloads all of it before first paint, on every load, on a phone.

The sharpest edge is `workspace_builder_state`. One jsonb column holds every App Builder app *and every record* for a company. It already measures **525 kB for a test company with ten apps**, and 264 kB for another. It is read whole at startup and — as your own code comment in `wb-intake-submit.js` puts it — “*append one record is really rewrite everything.*” That document grows without bound, and both its read and its write cost scale with total company history rather than with what changed.

FIX

Split startup into a bounded shell tier that gates first paint and a per-route tier that pages. Separately, move App Builder *records* out of the document into rows with `workspace_id`, RLS and an index — see recommendation 08.

03

SCALE

Rate limiting

Rate limiting is a Map in lambda memory

`_lib/rate-limit.js` holds counters in module scope. On Vercel the effective limit is the configured number multiplied by live instances, and it resets on every cold start. That makes the published numbers advisory rather than real for portal password attempts, invite-token lookup, proposal token lookup and address suggestions.

Your intake code already documents this honestly. The gap is that three other endpoints rely on it without a second layer.

FIX

A Postgres-backed counter or Vercel KV. Same interface, one implementation swap in `consumeRateLimit`.

04

DEBT

API consistency

Two documented divergences are still carried in comments rather than resolved

`_lib/supabase-admin.js` contains an explicit “*UNRESOLVED DIVERGENCE*” block: two different `Authorization` header rules for the same service key are both live, via different call paths, and the merge deliberately preserved each rather than picking one. That is the right call at merge time and the wrong state to stay in — it means nobody currently knows which rule the deployed key actually needs.

Separately, four handlers still hand-roll their own try/catch instead of using `defineEndpoint`: `public-invite-lookup`, `public-proposal-open`, `public-proposal-respond` and `ringcentral-presence`. They miss the wrapper's uniform body cap and header guarantees.

FIX

Test the deployed key against both header shapes once, delete the losing branch, migrate the last four handlers.

05

COVERAGE

Release gate

Green CI still doesn't mean the app renders

`api/client-error.js` exists because a startup crash reached production and was found by a user seeing a blank page. The gap it documents is still open: 373 test files and a bundle-budget check, but nothing in `npm run check` ever loads the bundle in a browser.

Two of your own known-issues — the app-strip stylesheet conflict and the CSP-blocked jsPDF exports — were both found by rendering the deployed output *after* release, which is exactly the evidence that the gate has a hole in it.

FIX

One headless pass in the gate: load the built bundle, sign in as a test account, assert no console errors on three routes.

06

EFFICIENCY

SMS

Inbound SMS scans every contact in the company, twice

`sms-inbound.js` fetches all company contacts and matches the sender in JavaScript — then repeats the full fetch on the create-collision retry path. The comment “*contact counts are small*” is doing real load-bearing work.

FIX

Store a normalized E.164 column with an index and query it directly. Fold this into the workspace-safe SMS migration rather than doing it twice.

47,082 lines in one module has an audit cost, not just a maintenance one

Your recorded decision not to split `sic/main.js` in one move is right — the measured gain was small and the circular-import risk real. The consequence worth naming separately is that per-release security review of that file is not tractable by inspection, so the incremental-extraction plan is also your security-review plan.

FIX

Keep extracting on touch. Prioritise slices that render untrusted input or make authorization decisions, since those are what a reviewer most needs to be able to read.

Recommendations

Sequenced against the fact that you are pre-launch with pilot data. Everything here is cheaper now than it will ever be again — there is no migration burden, no customer to coordinate with, and no incident history to clean up.

01 Fix the tenancy guard's blind spot first

Everything
else
in this
report
is a
point-
in-
time
observation;
this is
the
control
that
keeps
the
property
true
after I
stop
looking.
Fix it
before
the
findings
it
should
have
been
catching.

02 Close portal session revocation and give guests a real identity

Both [s1](#) · [s2](#)

land
in the
same
two
files.
Status
re-
check
goes
in the
shared
endpoint
wrapper;
the
guest
id
goes
in the
session
payload.

03 Make the intake counters atomic

Lockout [s3](#)
and
submission
cap.
Both
are
the
same
read-
modify-
write,
both
fixed
by the
same
RPC
shape.

04 Put authentication on address suggestions

An [S4](#)

hour's
work,
and it
closes
the
only
endpoint
that
reads
no
session
at all.

05 Verify the portal session secret is set, then remove the fallback

Check [S6](#)

Vercel
production,
then
make
the
service-
key
substitution
throw
outside
development.

06 **Revoke the legacy anon grants and align the policy roles**

One [H1](#) · [H2](#) · [H3](#)

migration
covering
the
grant
revoke,
the
four
auth-
kernel
functions,
and
the
TO
publi
c
policies.
No
behaviour
change,
three
layers
restored.

01 Tier the startup path

A 02

bounded
shell
tier —
companies,
memberships,
workspaces,
roles,
permissions,
plugins
—
gates
first
paint.
Records
load
per
route,
paginated
server-
side.
This
is
also
the
mobile
fix
and
the
cold-
start
fix.

02 Decompose workspace_builder_state

App 02

definitions

can
stay a
document.

App

records

need

to be

rows

with

work

space

_id ,

RLS

and

an

index.

This

is the

highest-

leverage

change

available:

it

unblocks

App

Builder

at

scale,

lets

intake

submissions

be

accepted

without

rewriting

the

company

document,

and

makes

record-

level

permissions

and

record-
level
history
possible
at all.

03 **Move rate limiting somewhere durable**

One 03 · S3 · S4

implementation

swap

behind

the

existing

cons

umeRa

teLim

it

interface.

04 **Finish or remove workspace SMS**

Decide S5 · 06

it. A

deployed

endpoint

held

shut

by a

constant

in a

different

module

will

eventually

be

opened

by

someone

who

doesn't

know

that's

what

they're

doing.

Tenant isolation as a live probe, not a snapshot diff

Your `decisions.md` already describes a rollback-only probe that impersonates a real non-admin identity and asserts zero foreign-tenant rows. Promote it from a documented one-off to a scheduled job against a seeded staging project. It is the only thing that proves policy *predicates* rather than policy *existence* — which is precisely the limit the tenancy script names about itself. It would find nothing today, and that is the right moment to install it.

Grant and posture drift detection

A scheduled check asserting: required environment variables present; `anon` holds no unexpected grants; no definer function is anon-executable; no function has a mutable search path; every tenant table has RLS with a policy. All five queries are in this audit and run in under a second. Wire them into the Production Guardian you already run six-hourly.

A browser check in the release gate

The `client-error` endpoint gives you the after-the-fact half of this already. The before-the-fact half is one headless pass over three routes. Given that two shipped defects were found this way manually, the automation pays for itself immediately.

Record history extended to App Builder records

`record_history` already covers Contacts, Quotes, Jobs and Tasks. Once records are real rows, extending it is cheap — and for companies handling insurance claims, a defensible “who changed this number, and when” ledger is a sales feature before it is a compliance one.

Portal guests as first-class objects

Fixing S2 properly produces a per-guest identity, and that unlocks what customers ask for next: per-guest activity trails, revoke-one-guest, an honest answer to “has the homeowner actually opened this”, and review threads that can't be impersonated.

A published security posture

You are selling multi-tenant SaaS to companies holding homeowner PII and insurance data. Complete RLS coverage, no anon-reachable privileged code, private buckets with short-lived signed URLs, and zero dependency CVEs is a better posture than is typical in this segment, and it is currently invisible to buyers. The Office-viewer data flow means you will be asked for a subprocessor list regardless — write the page while the answers are all good.

One framing note. The pilot-scale numbers cut both ways. They mean nothing in this report is currently on fire — there is no customer data at risk and no active abuse to respond to. They also mean the S-series findings are cheap right now and expensive later: every one of them is a change to an interface that no external integrator has bound to yet.

The single most valuable item is **O1**. Not because it represents exposure — it doesn't, I verified all seven tables — but because a green check that isn't checking anything is worse than no check at all. It is the one finding that makes the others harder to see.

Method — static review of all 26 API handlers, 3 Edge Functions and the shared request/auth libraries; escaping trace across every untrusted-input render path in src/; live verification against the Supabase production catalog covering advisors, role grants, RLS policy predicates, definer-function privileges and search paths, storage bucket visibility, and table sizes; npm audit across prod and dev trees. No writes were made to production. Findings not previously recorded in .ai/known-issues.md are marked as such; overlaps are noted inline.